ÍNDICE

ÍNDICE 1
Guía 3
2.1 Manual de configuración 3
2.1.1 Requisitos previos. 3
2.1.2 Instalación de Ubuntu 22.04.5 LTS. 3
2.1.3 Instalación de ROS 2 Humble. 3
2.1.4 Uso de Cyclone DDS 5
2.1.5 Crear el espacio de trabajo (workspace) 6
2.1.6 Conectar el Kobuki 8
2.1.7 Deshabilitar el autosuspend USB 9
2.1.8 Lanzar el nodo kobuki 10
2.1.9 Verificación de comunicación con el Kobuki 11
2.2 Control básico de desplazamiento 12
2.2.1 Aprendiendo de sensores 12
2.2.2 Creación de trayectorias y movimientos básicos 13
2.3 Integración del sensor Kinect (LiDAR) 16
2.3.1 Instalación del driver libfreenect 16
2.3.2 Instalación de Kinect ROS 2 18
2.3.3 Estructura del paquete de lanzamiento 22
2.4 Procesamiento de la señal LiDAR 23
2.4.1 Conversión de nube de puntos 3D a LaserScan 2D 23
2.4.2 Finalización de conversión, árbol de transformadas y verificación de escaneo 25
2.4.3 Filtrado de la señal LaserScan 28
2.4.4 Verificación visual en RViz2 29
2.5 Implementación de SLAM 31
2.5.1 Instalación de slam_toolbox 31
2.5.2 Configuración del mapper_params_online_async.yaml 31

2.5.3 Fusión de odometría e IMU con robot_localization 32
2.5.4 Launch file de mapeo (slam.launch.py) 33
2.5.5 Proceso de mapeo 35
2.5.6 Guardar el mapa 36
2.6 Navegación Autónoma 36
2.6.1 ¿Qué es Nav2? 37
2.6.2 Instalación de Nav2 37
2.6.3 Configuración de nav2_params.yaml 38
III. RESULTADOS 44
V.BIBLIOGRAFIA 48
VI. ANEXOS 49

Guía

2.1 Manual de configuración

2.1.1 Requisitos previos.

Antes de comenzar con la configuración del entorno de desarrollo, es importante consultar la documentación oficial [1] y la guía de usuario del robot Kobuki [2] para familiarizarse con sus componentes y funciones.

2.1.2 Instalación de Ubuntu 22.04.5 LTS.

Primeramente, se debe descargar la imagen del sistema operativo Ubuntu 22.04.5 e instalarla en el equipo (ver Anexo A).

Una vez instalado el sistema operativo, se deben actualizar los paquetes del sistema.

Bash:

```
sudo apt update && sudo apt upgrade -y
```

2.1.3 Instalación de ROS 2 Humble.

Antes de crear el espacio de trabajo, es necesario instalar ROS 2 Humble e instalaciones complementarias, es recomendable al menos revisar la guía oficial [3].

De forma resumida los comandos a usar serán los siguientes:

Configurar locales

Bash:

```
sudo apt install -y locales
```

```
sudo locale-gen en_US en_US.UTF-8

sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
```

Agregar el repositorio de ROS 2

Bash:

```
sudo apt install -y software-properties-common curl

sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \

-o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) \

signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \

http://packages.ros.org/ros2/ubuntu \

$(. /etc/os-release && echo $UBUNTU_CODENAME) main" \

| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Instalar ROS 2 Humble completo

Bash:

```
sudo apt update

sudo apt install -y ros-humble-desktop
```

Agregar ROS 2 al entorno

Bash:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

source ~/.bashrc
```

Verificar que la instalación fue exitosa:

Bash:

```
ros2 --version
```

2.1.4 Uso de Cyclone DDS

Se recomienda usar Cyclone DDS en lugar del middleware por defecto (FastDDS), esto ayuda a evitar problemas de sincronización de timestamps observados posteriormente en la localización (AMCL):

Bash:

```
sudo apt install ros-humble-rmw-cyclonedds-cpp
```

```
echo "export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp" >> ~/.bashrc
```

```
source ~/.bashrc
```

Para verificar si estamos utilizando Cyclone DDS se ejecuta el siguiente comando.

Bash:

```
echo $RMW_IMPLEMENTATION
```

Y para confirmar la consola debe responde de la siguiente manera, es así como se sabe que estamos usando Cyclone DDS

Bash:

```
rmw_cyclonedds_cpp.
```

2.1.5 Crear el espacio de trabajo (workspace)

Se creará la carpeta del proyecto (espacio de trabajo) donde se alojarán las carpetas y archivos del mismo:

Bash:

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws
```

Ahora se clonarán los repositorios de GitHub necesarios para guardarlos en la carpeta src:

Bash:

```
cd ~/ros2_ws/src
git clone https://github.com/kobuki-base/kobuki_ros.git
git clone https://github.com/kobuki-base/kobuki_core.git
git clone https://github.com/kobuki-base/velocity_smoother.git
git clone https://github.com/kobuki-base/cmd_vel_mux.git
```

Los paquetes correspondientes al Kinect (kinect_ros2, libfreenect, depthimage_to_laserscan) se instalan más adelante, en la etapa de integración de sensores.

Ahora se va a inicializar y actualizar rosdep, esto servirá para actualizar las dependencias.

Bash:

```
sudo rosdep init
rosdep update
```

Enseguida se procede a instalar las dependencias faltantes del espacio de

trabajo (workspace).

Bash:

```
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
```

Ahora se instalan las dependencias adicionales de ROS 2

Bash:

```
sudo apt update
sudo apt install -y ros-humble-diagnostic-updater \
ros-humble-joint-state-publisher \
ros-humble-ros-testing
```

Y finalmente se compila el workspace.

Bash:

```
cd ~/ros2_ws
colcon build
```

Enseguida se debe agregar el workspace para que el entorno de ROS2 siempre lo encuentre.

Bash:

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Ahora para confirmar que los paquetes se instalaron correctamente, se ejecuta el siguiente comando y se deberá ver como la ilustración 1.

Bash:

```
ros2 pkg list | grep kobuki
```

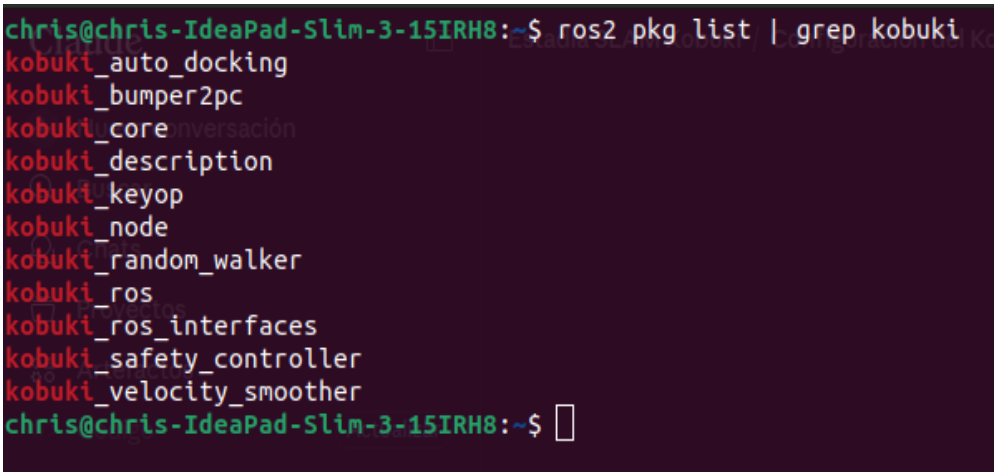


Ilustración 1. Verificación de paquetes instalados.

2.1.6 Conectar el Kobuki

Ahora se va a conectar el robot Kobuki a la computadora mediante un cable USB como se muestra en la ilustración 2, para después ejecutar los siguientes comandos, esto servirá para verificar y configurar la conexión entre ambos hardware.

El siguiente comando sirve para verificar si el sistema reconoce el dispositivo USB.

Bash:

```
ls /dev/ttyUSB*
```

Si el dispositivo aparece (generalmente lo hace como /dev/ttyUSB0), agregar el usuario al grupo dialout para otorgar permisos de acceso al puerto serie.

Bash:

```
sudo usermod -a -G dialout $USER
```

Este comando sirve para otorgar permisos de acceso al puerto serial, este comando es uno de los pasos más importantes, puesto que, sin esto, el sistema

no podrá comunicarse con el robot; cabe mencionar que se necesita cerrar sesión y volver a iniciarla (o reiniciar el equipo) para que el cambio surta efecto.

Una vez reiniciado el equipo, se usa el siguiente comando para verifica que el usuario ya pertenece al grupo dialout.

Bash:

```
groups $USER
```

El siguiente comando confirma que el dispositivo tiene permisos de lectura/escritura para el grupo dialout.

Bash:

```
ls -la /dev/ttyUSB0
```



Ilustración 2. Robot Kobuki conectado.

2.1.7 Deshabilitar el autosuspend USB

Este es un paso importante ya que Ubuntu por defecto puede suspender automáticamente los dispositivos USB tras un periodo de inactividad, lo cual provoca desconexiones inesperadas del Kobuki y del Kinect durante su uso de los mismo, es por ello que es necesario deshabilitar esta función.

Para deshabilitar el autosuspend de forma permanente, y si se desea crear o editar

el archivo de configuración esto si en un futuro se desea cambiar o regresar la configuración básica, se deberá ejecuta el siguiente comando.

Bash:

```
sudo nano /etc/modprobe.d/usbcore.conf
```

Después se agrega la siguiente línea y se debe guardar

Bash:

```
options usbcore autosuspend=-1
```

Para guardar el archivo, Presiona Ctrl + O para guardar y luego Enter para confirmar el nombre del archivo, Enseguida Presiona Ctrl + X para salir del editor.

Finalmente, para aplicar los cambios, utiliza el el siguiente comando.

Bash:

```
sudo update-initramfs -u
```

Por último, es necesario reiniciar el equipo para que la configuración surta efecto.

2.1.8 Lanzar el nodo kobuki

Con los permisos configurados y el autosuspend deshabilitado, es el momento preciso para lanzar el nodo principal del Kobuki, para ello se ejecutan el siguiente comando.

Bash:

```
ros2 launch kobuki_node kobuki_node-launch.py
```

Esto permitirá que los sensores y odometría del kobuki inicien junto con todas sus integraciones necesarias para su funcionamiento.

Si este comando no funciona posiblemente tengas que compilar paquete

kobuki_node de forma individual, para esto se ejecutan las siguientes líneas de comando.

Bash:

```
cd ~/ros2_ws
colcon build --packages-select kobuki_node
source ~/.bashrc
```

Y se vuelve a ejecutar exactamente el mismo comando.

Bash:

```
ros2 launch kobuki_node kobuki_node-launch.py
```

2.1.9 Verificación de comunicación con el Kobuki

Para confirmar que existe comunicación entre la computadora y el robot, se deberá ejecutar el siguiente comando:

Bash:

```
ros2 topic echo /odom
```

Se verán datos de posición (x, y), orientación (quaternion) y velocidades actualizándose en tiempo real.

2.2 Control básico de desplazamiento

Una vez finalizada la conexión y verificación de envío de datos entre el robot y la computadora, es aquí cuando la lectura de la documentación oficial [1] cobra sentido pues, este subtema va aunar por la estructura general del robot, sus componentes, conexiones, batería, comunicación, sensores básicos y comandos básicos de movimiento, así como diseñar trayectorias.

2.2.1 Aprendiendo de sensores

Los sensores del robot Kobuki son fundamentales para su operación, ya que permiten para garantizar una navegación autónoma segura y precisa, midiendo el entorno y el estado interno del aparato, es por ello que es es de suma importancia conocerlos, por ello siguiente comando sirve para ver la lista de los sensores que utiliza el robot.

```
ros2 topic list
```

Este comando muestra los tópicos activos de comunicación, estos tópicos incluyen comandos de control, odometría, datos sensoriales y transformadas, como ejemplo se muestran los tópicos que se utilizaran para este proyecto.

- `/commands/velocity – para mover el robot`
- `/odom – odometría`
- `/sensors/imu_data – IMU`
- `/events/bumper – sensores de choque`
- `/tf – transformadas`

En este primer ejercicio, se hará uso del siguiente comando:

```
ros2 topic echo /odom
```

Este comando sirve para mostrar en tiempo real, en la terminal los mensajes de odometría que publica un robot, es decir permite ver la posición, la orientación y la velocidad del robot mientras este se desplaza.

Una vez verificado el funcionamiento de sensores de seguridad, se pasará al siguiente punto el cual se enfocará en realizar los primeros movimientos básicos y trayectorias.

2.2.2 Creación de trayectorias y movimientos básicos

En este apartado se desarrollarán rutinas de programación en Python (rclpy) para el control de movimiento del robot Kobuki; se implementarán secuencias como lo son el avanzar una distancia aproximada, girar cierto ángulo, detenerse ante obstáculos y/o evitarlos y ejecutar trayectorias con cierto nivel de complejidad para el robot.

Para el primer movimiento, se programará avanzar la distancia de un metro, para ello se deben seguir los siguientes pasos:

- Ubicación en el espacio de trabajo:

  Bash:

  ```
  cd ~/ros2_ws/src
  ```

- Creación del paquete en Python:

  Bash:

  ```
  ros2 pkg create kobuki_trajectories --build-type ament_python --dependencies rclpy geometry_msgs
  ```

- Desarrollo del nodo de la trayectoria:

  Bash:

```
nano
~/ros2_ws/src/kobuki_trajectories/kobuki_trajectories/mo
ver_metro.py
```

Nota: La lógica del nodo publica mensajes de tipo geometry_msgs/msg/Twist en el tópico /commands/velocity a una frecuencia fija hasta cumplir el tiempo estimado para recorrer un metro. El código fuente se encuentra disponible en el Anexo B.

- Configuración de puntos de entrada (setup.py):

  Se edita el archivo setup.py para registrar el ejecutable del nodo en la sección console_scripts

  Bash:

  ```
  nano ~/ros2_ws/src/kobuki_trajectories/setup.py
  ```

  Busca la sección entry_points y déjala así:

  ```
  python

  entry_points={

      'console_scripts': [

          'mover_metro                                        =
  kobuki_trajectories.mover_metro:main',

      ],

  },
  ```

  5.Guarda y compila:

  Bash:

  ```
  cd ~/ros2_ws
  ```

```
colcon build --packages-select kobuki_trajectories

source ~/.bashrc
```

Ahora con el nodo de lanzamiento del Kobuki corriendo en una terminal, en otra terminal ejecuta:

Bash:

```
ros2 run kobuki_trajectories mover_metro
```

Listo finalmente has creado la primera trayectoria para el robot Kobuki, si se desea ver demostraciones de las trayectorias y movimientos puede ver las demostraciones en YouTube (ver Anexo C).

El movimiento del robot consiste en trazar una línea recta una distancia exacta (por defecto 1.0 metro) con control en lazo cerrado, es decir hará la trayectoria independientemente del tiempo que le tome recorrerla.

La manera en que funciona, es que el sistema captura las coordenadas cartesianas iniciales (X_0, Y_0) desde /odom y calcula en tiempo real la distancia euclidiana $\sqrt{(\mathrm{x}-\mathrm{x}_0)^2+(\mathrm{y}-\mathrm{y}_0)^2}$ hasta superar el umbral configurado, frenando los motores inmediatamente.

Ahora bien, una vez creada la primera trayectoria, es buen momento para comenzar a experimentar con movimientos y trayectorias cada vez más complejas, es por eso que se puede consultar y probar las más de 12 trayectorias de pruebas directamente en el directorio src/kobuki_trajectories/kobuki_trajectories en el repositorio del proyecto [4].

2.3 Integración del sensor Kinect (LiDAR)

El siguiente gran paso es montar el sensor LiDAR, específicamente en este proyecto no se usará un LiDAR tradicional; en su lugar se hará uso de un Kinect versión 1 de la consola Xbox 360, la cual es una cámara de profundidad RGB-D, pero para los propósitos del proyecto es más que suficiente pues esta cámara de profundidad es capaz extraer un "scan" 2D de su nube de puntos 3D para usarlo como si fuera un LiDAR. Si se desea conocer más acerca del Kinect para aplicaciones de mapeo en interiores puede consultar el articulo [5] una vez aclarado este punto, se procede con la instalación del driver.

2.3.1 Instalación del driver libfreenect

1. Primero, se instala el driver con el siguiente comando:

Bash:

```
sudo apt install libusb-1.0-0-dev libudev-dev freeglut3-dev
libxmu-dev libxi-dev
```

2. Después, se instalan y se clonan los paquetes de libfreenect con los siguientes comandos:

Bash:

```
cd ~/ros2_ws/src
git clone https://github.com/OpenKinect/libfreenect.git
cd libfreenect
mkdir build && cd build
cmake ..
make
sudo make install
```

3. Se agregan reglas udev para que el sistema reconozca al Kinect:

Bash:

```
sudo cp ~/ros2_ws/src/libfreenect/platform/linux/udev/51-kinect.rules /etc/udev/rules.d/

sudo udevadm control --reload-rules

sudo udevadm trigger
```

4. Ahora se actualiza el ldconfig caché con ayuda del siguiente comando:
Bash:

```
sudo sh -c 'echo "/usr/local/lib" > /etc/ld.so.conf.d/libfreenect.conf'

sudo ldconfig
```

5. El siguiente paso es conectar el Kinect al Kobuki mediante el cable USB y el alimentar el Kinect con el puerto de pines de 12v y 1.5A que posee el Kobuk como se muestra en la ilustración 3, enseguida se abre; enseguida se verifica si el sistema es capaz de detectar al Kinect con ayuda del siguiente comando:

Bash:

```
lsusb | grep -i Xbox
```



Ilustración 3. Kinect conectado a robot Kobuki.

6. El siguiente paso es abrir libfreenect para verificar si está funcionando y si todo se instaló correctamente, se podrá ver una interfaz como la que se muestra en la ilustración 4.

Bash:

```
freenect-glview
```

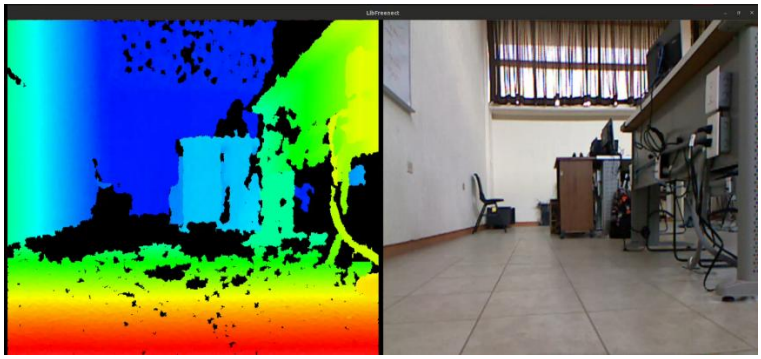


Ilustración 4. LibFreenect funcionando.

2.3.2 Instalación de Kinect ROS 2

Para la incorporación de la cámara de profundidad Microsoft Kinect v1 al ecosistema ROS 2 Humble, se requiere la compilación de sus drivers compatibles

y el paquete de conversión a escaneo láser sintético. El procedimiento técnico de configuración se detalla a continuación:

1. Primero, se instala el driver principal Kinect_ros2 y el nodo depthimage_to_laserscan para la generación del láser 2D, esto se hará con el siguiente comando:

Bash:

```
cd ~/ros2_ws/src
```

```
git clone https://github.com/fadlio/kinect_ros2.git
```

2. Después se clona el depthimage_to_laserscan, con el siguiente comando:

Bash:

```
git clone https://github.com/ros-perception/depthimage_to_laserscan.git
```

3. Se instala la dependencia adicional:

Bash:

```
sudo apt update
```

```
sudo apt install ros-humble-depth-image-proc
```

4. Una vez instalado y clonados los programas, se procede a compilar todo junto:

Bash:

```
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build
source install/setup.bash
```

5. Ahora se verifican si los paquetes están disponibles:

Bash:

```
ros2 pkg list | grep kinect
ros2 pkg list | grep depthimage
```

Y deberá aparecer:

Bash:

```
kinect_ros2
depthimage_to_laserscan
```

6. El último paso es verificar que el Kinect publica datos correctamente, para esto; primero se debe conectar el Kinect (USB + alimentación 12V al Kobuki), posteriormente se inicializa el nodo de la nube de puntos 3D.

Bash:

```
ros2 launch kinect_ros2 pointcloud.launch.py
```

Este comando inicializa el driver junto con la interfaz gráfica RViz2, en la cual se visualiza la nube de puntos 3D en el tópico /kinect/points y fixed frame en base_footprint. (ver Figura 3.1).

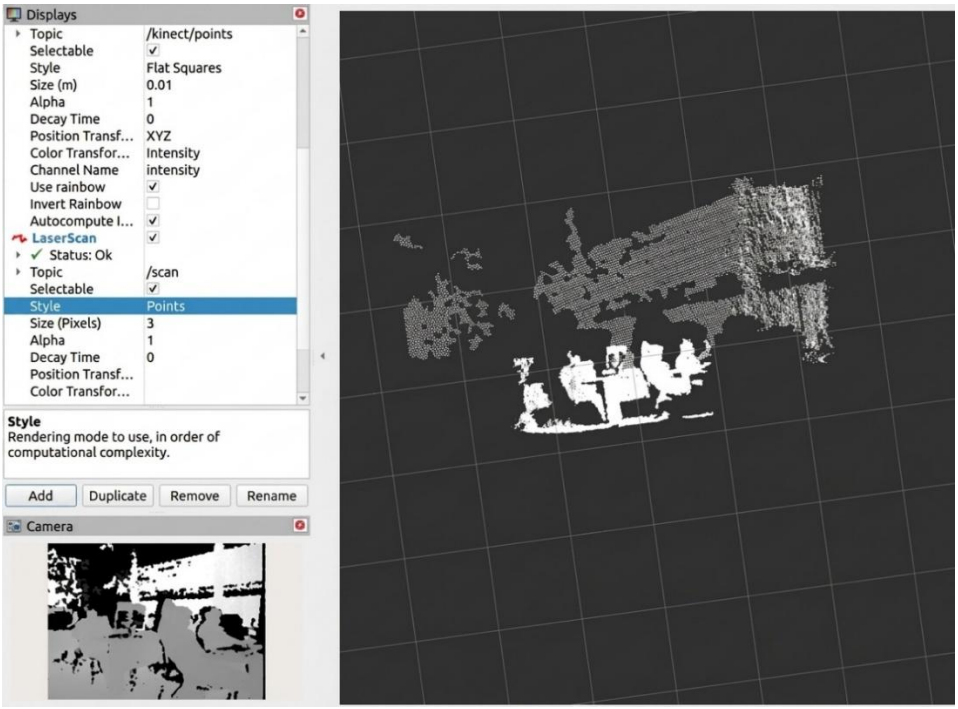


Ilustración 5. Nube de puntos 3D en RViz2.

Nota: RViz2 es la herramienta de visualización de ROS 2 y viene preinstalada con ros-humble-desktop. Para abrirla manualmente:

Bash:

```
rviz2
```

Nota sobre la ejecución para navegación: Si bien el archivo. launch es ideal para la inspección visual inicial, se identificó que genera un nodo duplicado que provoca problemas de sincronización de marcas de tiempo (timestamps) durante la localización con AMCL. Por ello, para la etapa de procesamiento y generación de mapa 2D, se debe ejecutar el nodo directamente restringiendo la frecuencia a 10 Hz (esto se pude configurar) para optimizar recursos:

Bash:

```
ros2 run kinect_ros2 kinect_ros2_node --ros-args \ --param fps:=10
```

```
\ --param depth_registration:=false \ --remap __ns:=/kinect
```

7. Ahora con este nuevo comando se procede a la verificación tópicos y frecuencia de actualización, para ello, en una terminal secundaria, se valida la presencia del flujo de datos en tiempo real y la estabilidad del sensor:

Bash:

```
ros2 topic list | grep kinect
```

```
ros2 topic hz /kinect/depth/image_raw
```

Este comando obtiene una frecuencia constante de aproximadamente 10 Hz en el tópico /kinect/depth/image_raw confirmando que el sensor se encuentra listo para integrarse con el nodo de conversión a láser 2D.

2.3.3 Estructura del paquete de lanzamiento

Con el fin centralizar los archivos de configuración y scripts de inicio (launch files) del Kobuki y sus sensores, se creó un paquete dedicado en el espacio de trabajo mediante el siguiente proceso:

Bash:

```
cd ~/ros2_ws/src
```

```
ros2 pkg create kobuki_launch --build-type ament_python --dependencies rclpy
```

```
mkdir -p ~/ros2_ws/src/kobuki_launch/launch
```

Este directorio se alojarán los archivos de parámetros. yaml (como la configuración del escaneo láser) y los scripts de lanzamiento unificados del

sistema que se aplicarán en las siguientes secciones de este reporte.

2.4 Procesamiento de la señal LiDAR

El sensor Kinect funciona correctamente e inclusive ya es capaz de publicar una imagen de puntos 3D, sin embargo, es necesario transformar esa nube de puntos 3D en un LaserScan 2D al igual que configurar ciertos parámetros que procesa la señal del Kinect ya que como se menciona en el artículo [5], el Kinect tiene errores sistemáticos que solo se pueden eliminar mediante un procedimiento de calibración adecuado, es por ello que a continuación se detallan esos puntos.

2.4.1 Conversión de nube de puntos 3D a LaserScan 2D

Se mencionó que es necesario transformar la nube de puntos 3D en una de tipo LaserScan, esto es importante para poder reducir el costo computacional, aprovechar los algoritmos 2D maduros y poder compatibilizar interfaces estándar en frameworks de ROS 2, para ello en ROS 2 Humble existe un paquete llamado depthimage_to_laserscan; lo que hace a este paquete tan interesante es que toma una imagen de profundidad en 3D como la que publica el Kinect y la transforma en un mensaje de tipo LaserScan de 2D lo que permite simular un sensor láser tradicional.

El siguiente diagrama (ilustración 6) muestra cómo se transforma una nube de puntos 3D a un LaserScan 2D, a la izquierda de la imagen, se puede observar que la cámara Kinect en el Kobuki capta una nube de puntos 3D completa mientras que, a la derecha, se observa cómo el software realiza un corte transversal o franja horizontal el cual se muestra como un plano rojo y esta termina proyectando esos puntos en un plano 2D, creando de esta manera el LaserScan que más adelante se usará para el algoritmo de SLAM.

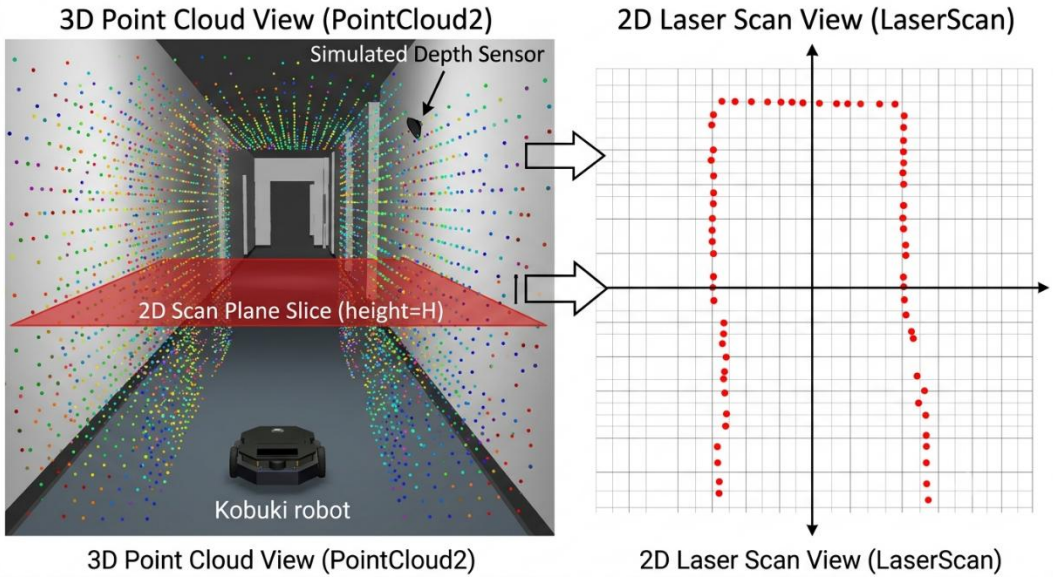


Ilustración 6. Diagrama de conversión de datos sensoriales (3D a 2D).

A pesar de que el kinect es excelente para tareas de mapeo aún sigue contando con ciertas limitaciones específicamente cuando se obtienen mediciones a largas distancias, ya que estas, afectan la calidad de los datos los cuales se degradan debido al ruido y a la baja resolución de las mediciones de profundidad., por ello es recomendable adquirir los datos en una distancia de 1–3 metros del sensor [5]; es aquí cuando entra en escena nuestro archivo "laserscan_config.yaml" (el cual se puede encontrar en la ruta src/kobuki_launch/config en el repositorio del proyecto [4]), este archivo principalmente sirve para configurar los parámetros del Kinect, a continuación se detalla que hace exactamente cada configuración y como ayuda al kinect a generar mejores imágenes.

`scan_time: 0.11` <- Esta línea define el intervalo entre escaneos (~9-11 Hz) para estabilizar la publicación de los datos.

`range_min: 0.5` <- Límite inferior: ignora lecturas a menos de 50 cm por ceguera cercana del sensor, es decir detecta obstáculos desde 50 cm en adelante (evita

choques cercanos).

`range_max: 4.0` <- Límite superior: acota la distancia máxima para eliminar ruido de fondo, realiza el corte a los 4 metros.

`scan_height: 3` <- Número de filas verticales de la imagen usadas para generar el plano 2D.

`approximate_sync: true` <- Permite la sincronización aproximada de estampas de tiempo (timestamps).

`queue_size: 2` <- Reduce la cola de mensajes de 10 a 2 para descartar cuadros viejos y liberar CPU.

Cabe mencionar que estás configuraciones fueron diseñadas para un entorno especifico y con hardware especifico con sus limitaciones, por esto es recomendable que el usuario realice sus propias configuraciones de acuerdo a su entorno y sus herramientas disponibles.

2.4.2 Finalización de conversión, árbol de transformadas y verificación de escaneo

Una vez configurado el archivo de parámetros, se ejecuta el nodo de conversión integrando el archivo YAML esto para verificar que el mensaje de tipo LaserScan se publique con la tasa de refresco requerida, pero primero se debe de crear la carpeta de configuración en donde se almacenarán los archivos de configuraciones de parámetros, para ello se ejecuta el siguiente comando:

Bash:

```
mkdir -p ~/ros2_ws/src/kobuki_launch/config
```

Ahora simplemente se pueden copiar y pegar los archivos que se encuentran en src/kobuki_launch/config del repositorio del proyecto [4]; después en una nueva terminal se ejecuta el nodo de conversión:

Bash:

```
ros2 run depthimage_to_laserscan depthimage_to_laserscan_node --ros-args \
  --params-file ~/ros2_ws/src/kobuki_launch/config/laserscan_config.yaml \
  -r depth:=/kinect/depth/image_raw \
  -r depth_camera_info:=/kinect/depth/camera_info \
  -r scan:=/scan
```

El siguiente paso es configurar el árbol de transformadas, pero antes de configurarlas, es importante entender que árbol de transformadas (o árbol de *tf*) es una estructura de datos jerárquica usada en robótica para conectar y calcular las posiciones espaciales (rotación y traslación) entre diferentes partes de un robot y su entorno en tiempo real [6].

Para continuar con la configuración de transformadas TF, en terminales individuales se colocarán los siguientes comandos:

Bash:

```
# Terminal 1. TF 1: Posición del Kinect sobre el robot
ros2 run tf2_ros static_transform_publisher \
  0.1 0 0.3 0 0 0 base_footprint kinect_depth

# Terminal 2. TF 2: Frame interno del driver
ros2 run tf2_ros static_transform_publisher \
```

```
  0 0 0 0 0 0 kinect_depth camera_depth_frame
```

Terminal 3. TF 3: Requerida por Nav2

```
ros2 run tf2_ros static_transform_publisher \
  0 0 0 0 0 0 base_footprint base_link
```

Oh bien ejecutar todas en una en una sola terminal usando & para ejecutar en segundo plano, de la siguiente manera:

Bash:

```
ros2 run tf2_ros static_transform_publisher 0.1 0 0.3 0 0 0 base_footprint kinect_depth &

ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 kinect_depth camera_depth_frame &

ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 base_footprint base_link
```

Y finalmente se monitorea la frecuencia en una terminal secundaria, para ello se ejecutan el siguiente comando:

Bash:

```
ros2 topic hz /scan
```

Y se mostrará una publicación estable cercana a los 10 Hz que confirma que la conversión del plano 2D se ejecuta en tiempo real sin saturación del sistema.

2.4.3 Filtrado de la señal LaserScan

A pesar de acotar el rango con el archivo YAML, el escaneo resultante del Kinect suele presentar "puntos fantasmas" o lecturas erráticas en los extremos del campo de visión debido al reflejo infrarrojo en estructuras laterales o imperfecciones del suelo; por ello para garantizar la consistencia del mapa en el proceso de SLAM, es indispensable aplicar un nodo de filtrado secundario.

Así es que se desarrolló un nodo personalizado en Python (rclpy) que se suscribe al tópico /scan, procesa el arreglo de distancias (ranges) aplicando un filtro de rango angular y eliminación de valores inf/NaN, y reinterpreta los datos hacia un nuevo tópico limpio.

A continuación, se detalla como compilarlo e integrarlo:

1. Copiar el archivo y pegar el archivo filtro_laserscan.py (el cual se ubica en la ruta src/kobuki_trajectories/kobuki_trajectories del repositorio) en el entorno de trabajo que se creó anteriormente.

2. Se registra el punto de entrada en el archivo setup.py con ayuda de la siguiente línea de código:

   ```
   'filtro_laserscan                                                    =
   kobuki_trajectories.filtro_laserscan:main',
   ```

3. Finalmente se recompila el paquete y se ejecuta el nodo:

   Bash:

   ```
   cd ~/ros2_ws

   colcon build --packages-select kobuki_trajectories

   source install/setup.bash

   ros2 run kobuki_trajectories filtro_laserscan
   ```

2.4.4 Verificación visual en RViz2

Para validar la calidad del escaneo filtrado respecto al chasis del robot, se inicializa la herramienta RViz2 con el comando:

Bash:

```
rviz2
```

Enseguida se configura la escena en el panel izquierdo de los paneles (Displays) los paneles deben ser configurados del siguiente modo:

- Fixed Frame: En la parte superior de la lista (Global Options), cambia el campo Fixed Frame a base_footprint.
- Agregar el visualizador del láser: Haz clic en el botón Add (abajo a la izquierda) y selecciona el módulo LaserScan.
- Asignar el tópico filtrado: Despliega las opciones del LaserScan recién agregado, busca la propiedad Topic y selecciona /scan_filtrado (o /scan si vas a comparar el escaneo antes de filtrar).

Para verificar si se ha hecho correctamente el procedimiento, en lugar de ver la pared o las sillas completas en 3D, verás un contorno o silueta de puntos ordenados en un solo nivel de altura alrededor de la base del robot (base_footprint) como se muestra en la ilustración 7, esos puntos se comportan y se visualizan exactamente igual a como lo haría un sensor LiDAR 2D físico (como un Hokuyo o RPLiDAR).

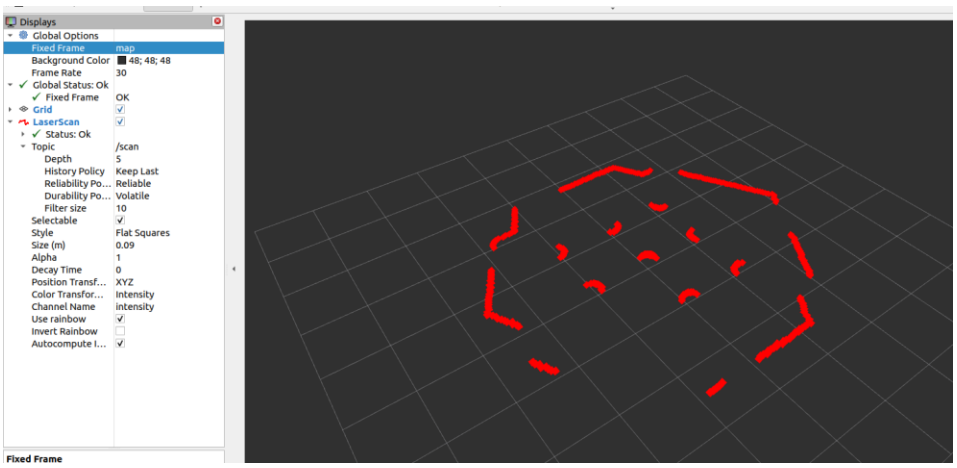


Ilustración 7. Proyección y verificación del patrón de escaneo bidimensional (LaserScan) en RViz2 en Simulador Gazebo.

2.5 Implementación de SLAM

Una vez que el sistema es capaz de recolectar información en 2D el siguiente paso es la generación autónoma de mapas del entorno; para ello se implementó el algoritmo SLAM (Simultaneous Localization and Mapping), el cual le permite al robot construir un mapa de un entorno desconocido mientras simultáneamente estima su propia posición dentro de él; en este proyecto se utilizó el paquete slam_toolbox, disponible para ROS 2 Humble, en lugar de Héctor SLAM que originalmente se contempló, ya que este último no cuenta con una versión oficial portada a ROS 2.

2.5.1 Instalación de slam_toolbox

Para instalar el paquete slam_toolbox, se debe ejecutar el siguiente comando:

Bash:

```
sudo apt install ros-humble-slam-toolbox
```

Y para verificar que el paquete se ha instalado, se debe ejecutar el siguiente comando:

Bash:

```
ros2 pkg list | grep slam
```

Y debe aparecer en la lista: `slam_toolbox.`

2.5.2 Configuración del mapper_params_online_async.yaml

El paquete slam_toolbox requiere de un archivo de parámetros para poder controlar el comportamiento del algoritmo de mapeo; Este archivo se encuentra en la ruta src/kobuki_launch/config/ que también está disponible en el repositorio del

proyecto [4]; a continuación, se describen los parámetros más importantes que contiene el archivo:

- odom_frame: odom — Frame de odometría usado por el algoritmo
- base_frame: base_footprint — Frame base del robot
- scan_topic: /scan_filtrado — Tópico del LaserScan filtrado
- resolution: 0.05 — Resolución del mapa en metros por celda
- max_laser_range: 4.0 — Rango máximo del láser, consistente con laserscan_config.yaml
- odometry_variance_penalty: 2.0 — Reduce la confianza en la odometría para que el algoritmo dependa más del LaserScan, evitando mapas diagonales
- loop_search_maximum_distance: 3.0 — Distancia máxima para buscar cierres de bucle
- loop_match_minimum_response_fine: 0.45 — Umbral mínimo de correlación para aceptar un cierre de bucle

2.5.3 Fusión de odometría e IMU con robot_localization

El robot Kobuki cuenta con varios sensores y sistemas de posicionamiento relativo, como lo es la odometría, el cual es el método para calcular la posición y orientación de un robot móvil usando sensores internos (como codificadores en las ruedas o unidades inerciales) para medir el desplazamiento desde un punto de inicio, sin embargo posee una gran desventaja, la cual es la derivada acumulativa (drift) los cuales son pequeños errores de medición que se suman paso a paso y provocan que la posición estimada se aleje de la realidad conforme pasa el tiempo; es por ello que se implementó un filtro de Kalman extendido (EKF) [7], mediante el paquete robot_localization, el cual fusiona la odometría de ruedas

(/odom) con los datos de la IMU (/sensors/imu_data), produciendo así una odometría más precisa y estable en el tópico /odom_filtered.

Para instalar el paquete que se utilizará, se hace uso del siguiente comando:

Bash:

```
sudo apt install ros-humble-robot-localization
```

El archivo de configuración ekf_config.yaml se encuentra en src/kobuki_launch/config del repositorio [4]. Los parámetros más importantes son:

- frequency: 30.0 — Frecuencia de publicación de la odometría fusionada en 30 Hz.
- two_d_mode: true — Restringe el cálculo al plano 2D.
- publish_tf: true — Publica la transformada odom → base_footprint mejorada.
- odom0: /odom — Fuente de odometría de ruedas.
- imu0: /sensors/imu_data — Fuente de datos inerciales.

Nota importante: Todos los valores numéricos en las matrices de covarianza deben ser decimales explícitos (0.0 en lugar de 0), de lo contrario el nodo EKF falla al parsear el archivo YAML.

2.5.4 Launch file de mapeo (slam.launch.py)

Para simplificar el lanzamiento de todos los nodos que son necesarios para hacer mapeo del entorno, se tuvo la idea de crear el archivo “slam.launch.py” el cual esta ubicado en la ruta src/kobuki_launch/launch del repositorio [4].

Este archivo lanza en orden los siguientes nodos:

- El driver del kobuki.
- El nodo del kinect.
- Las tres transformadas (TF).
- El nodo depthimage_to_laserscan.
- El filtro LaserScan.
- El filtro de Kalman.
- Nodo de slam_toolbox con RViz2.

Para ejecutar el mapeo completo, se diseñó un script de shell con el nombre “arranque_mapeo”, que se encuentra en Scripts_mapeo_&_navegacion del repositorio [4], el cual simplemente se puede copiar en la carpeta home y para ejecutarlo se deben seguir los siguientes pasos:

1. Se concede permisos de ejecución con el siguiente comando:

Bash:

```
chmod u+x arranque_mapeo.sh
```

2. Despues se escribe el siguiente comando, para arrancar “slam.launch.py:

Bash:

```
~/arranque_mapeo.sh
```

2.5.5 Proceso de mapeo

Con el sistema en ejecución y RViz2 mostrando el mapa en construcción (ilustración 8), se recomienda seguir una estrategia de recorrido en forma de U y/o circulo, esto quiere decir que se aplicará loop closure (ver Anexo D), esto es para maximizar la cobertura y la calidad del mapa; recomendaciones a seguir:

1. Colocar el robot al inicio del área a mapear, perfectamente alineado con el pasillo o la pared o algún punto de referencia.
2. Verificar en RViz2 que el LaserScan muestra las paredes horizontales antes de comenzar.
3. Avanzar pegado a la pared (o alguno punto de referencia) hasta el final del área.
4. Girar y regresar pegado a la pared (alguno punto de referencia).
5. Regresar al punto de inicio para cerrar el bucle (loop closure).

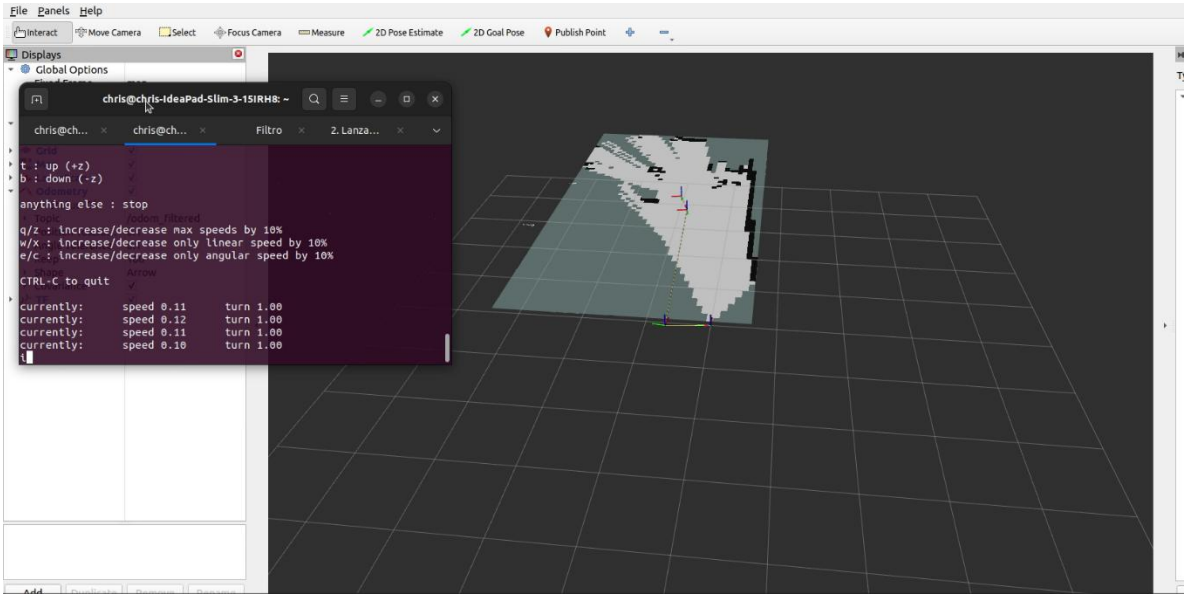


Ilustración 8. Construcción de mapa, en RViz2.

para que slam_toolbox tenga tiempo de procesar correctamente cada frame.

2.5.6 Guardar el mapa

Una vez completado el recorrido, el mapa se debe guardar en formato imagen, pues se utilizará Nav2/AMCL los cuales son compatibles para este tipo de formato; para guardar el mapa se ejecuta el siguiente comando:

Bash:

```
ros2 run nav2_map_server map_saver_cli -f ~/nombre_mapa
```

Esto generará dos archivos: nombre_mapa.pgm (imagen) y nombre_mapa.yaml (metadatos); en cuanto al archivo .pgm puede y es recomandable editarse manualmente con cualquier editor de imágenes para limpiar ruido o delimitar zonas restringidas.

Puedes consultar evidencia de creación de mapas en Anexo E.

2.6 Navegación Autónoma

Una vez generado el mapa del entorno, el siguiente paso es implementar la

navegación autónoma, es decir, lograr que el robot sea capaz de desplazarse de manera independiente desde un punto de inicio hasta un destino definido, esquivando obstáculos en tiempo real. Para ello se utilizó el stack de navegación Navigation2 (Nav2) [8], el cual es el framework estándar de navegación para ROS 2.

2.6.1 ¿Qué es Nav2?

Nav2 es el sistema de navegación autónoma estándar para robots móviles en ROS 2 (Robot Operating System 2). Su función principal es permitir que un robot se desplace de un punto a otro de forma segura, calculando rutas óptimas y esquivando obstáculos en tiempo real. [8] Sus componentes principales son:

- AMCL (Adaptive Monte Carlo Localization) — algoritmo de localización probabilística que usa un filtro de partículas para ubicar un robot en un mapa 2D conocido.[9]
- Planner Server — planifica la ruta óptima desde la posición actual hasta el destino.
- Controller Server — ejecuta la trayectoria planificada enviando comandos de velocidad al robot.
- Costmap 2D — genera mapas de costos local y global para detectar obstáculos y planificar rutas seguras.

2.6.2 Instalación de Nav2

Para instalar el stack de navegación Nav2 se ejecutan los siguientes comandos:

Bash:

```
sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup
```

Para instalar el stack de navegación Nav2 se ejecutan los siguientes comandos:

Bash:

```
ros2 pkg list | grep nav2
```

Deben aparecer varios paquetes con el prefijo nav2_.

2.6.3 Configuración de nav2_params.yaml

Nav2 requiere un archivo de parámetros para configurar el comportamiento de todos sus componentes. Este archivo se encuentra en src/kobuki_launch/config/nav2_params.yaml del repositorio [4]. A continuación, se describen los parámetros más relevantes:

AMCL:

- scan_topic: /scan_filtrado — usa el LaserScan filtrado para la localización.
- base_frame_id: base_footprint — frame base del robot.
- transform_tolerance: 0.5 — tolerancia en segundos para la sincronización de transformadas; este valor fue crítico para que AMCL funcionara correctamente con el driver del Kinect.
- max_particles: 2000 — número máximo de partículas del filtro.
- update_min_d: 0.2 — actualiza la localización cuando el robot avanza al menos 20 cm.

Controller Server:

- controller_frequency: 5.0 — frecuencia de control reducida para optimizar el uso de CPU.

- desired_linear_vel: 0.15 — velocidad lineal de navegación en m/s.
- transform_tolerance: 0.5 — tolerancia de transformadas para el controlador.

Costmap local:

- width: 3 y height: 3 — dimensiones del costmap local en metros.
- robot_radius: 0.18 — radio del robot para el cálculo de inflación de obstáculos.
- min_obstacle_height: 0.1 — ignora obstáculos por debajo de 10 cm (filtra el suelo).
- max_obstacle_height: 0.8 — ignora obstáculos por encima de 80 cm.

2.6.4 Launch file de navegación (navegacion.launch.py)

Para la navegación autónoma se creó el archivo navegacion.launch.py ubicado en src/kobuki_launch/launch del repositorio [4]. Este archivo lanza los siguientes componentes:

- El nodo EKF para la fusión de odometría
- El stack completo de Nav2 con AMCL y el mapa guardado
- El nodo relay que redirige los comandos de velocidad de Nav2 al tópico correcto del Kobuki
- RViz2 con la configuración de navegación

Nota importante: Nav2 publica los comandos de velocidad en el tópico

/cmd_vel, sin embargo, el Kobuki los espera en /commands/velocity. Para solucionar esto se utiliza el paquete topic_tools con un nodo relay:

Bash:

```
sudo apt install ros-humble-topic-tools
```

Este relay ya está integrado en el script de arranque.

Para ejecutar la navegación completa se utiliza el script arranque_navegacion.sh disponible en Scripts_mapeo_&_navegacion del repositorio [4]:

Bash:

```
chmod u+x arranque_navegacion.sh
```

```
~/arranque_navegacion.sh
```

2.6.5 Proceso de Navegación

Con el sistema en ejecución y el mapa cargado en RViz2 (ilustración 9), se deben seguir los siguientes pasos para iniciar la navegación autónoma:

1. Dar la pose inicial (2D Pose Estimate) — en RViz2, hacer clic en el botón "2D Pose Estimate" en la barra superior, luego hacer clic en el mapa en el punto donde está físicamente el robot y arrastrar para indicar la dirección hacia donde mira. Esto activa AMCL y establece la transformada map → odom.
2. Verificar la localización — tras dar la pose inicial, AMCL publicará la posición estimada del robot. Si las partículas convergen correctamente, el robot está localizado.
3. Dar el destino (2D Goal Pose) — hacer clic en "2D Goal Pose" en la barra superior de RViz2, luego hacer clic en el punto del mapa al que se desea

que el robot llegue y arrastrar para indicar la orientación final.

4. Observar la navegación — el robot planificará una ruta, comenzará a moverse y esquivará obstáculos dinámicos que detecte el Kinect en tiempo real.

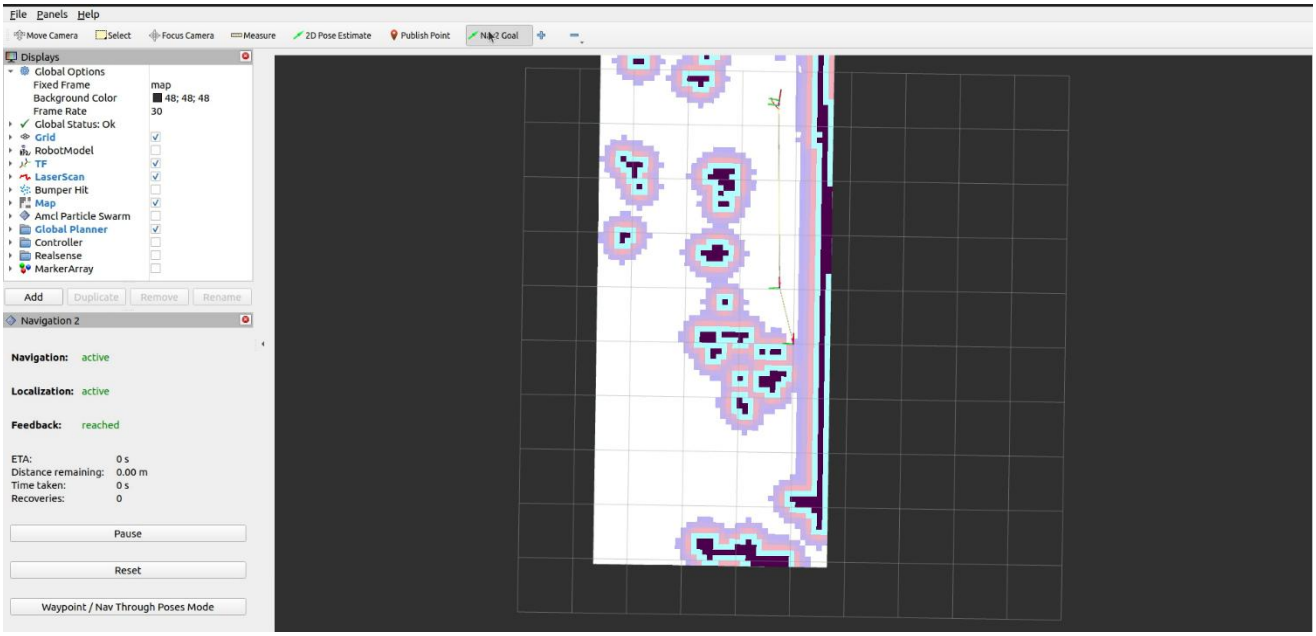


Ilustración 9. Funcionamiento de Nav2, en Rviz2.

Nota: El goal debe estar a una distancia mínima de aproximadamente 0.5 metros del robot, ya que Nav2 considera que distancias menores ya están dentro de la tolerancia y marca el goal como completado inmediatamente sin mover el robot.

2.6.5 Verificación en RViz2

Para visualizar correctamente el proceso de navegación en RViz2, se deben configurar los siguientes displays:

- Fixed Frame: map.

- Map → tópico /map — muestra el mapa guardado.
- LaserScan → tópico /scan_filtrado — muestra lo que el Kinect detecta en tiempo real.
- Path → tópico /plan — muestra la ruta planificada por Nav2.
- Costmap global → tópico /global_costmap/costmap — muestra el mapa de costos global.

Una navegación exitosa se verifica cuando el robot traza una ruta visible en RViz2, se desplaza siguiendo esa ruta y se detiene al llegar al destino (ilustración 10). Si encuentra un obstáculo no presente en el mapa, el costmap local lo detectará y el planner replanificará la ruta automáticamente.

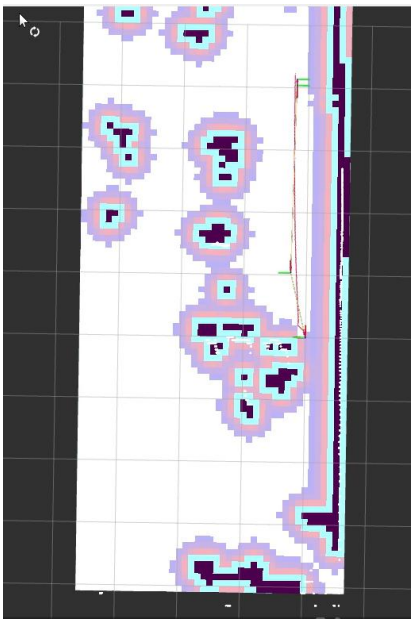


Ilustración 10. Ruta trazada con Nav2.

Se puede ver la evidencia del funcionamiento de la navegación autónoma del robot en Anexo F.

III. RESULTADOS

3.1. Control de desplazamiento

Las pruebas de control de desplazamiento se realizaron en dos modalidades: control en lazo abierto mediante temporización y control en lazo cerrado mediante odometría. Los resultados demostraron una mejora significativa en la precisión al incorporar retroalimentación de la odometría.

En el control en lazo abierto, el robot ejecutaba movimientos basados únicamente en el tiempo de ejecución del comando de velocidad, sin verificar la posición real alcanzada. Este método produjo errores variables dependientes de la superficie y el estado de la batería, con desviaciones de hasta 15-20 cm en desplazamientos de 1 metro.

Con el control en lazo cerrado mediante odometría, el robot monitorea continuamente su posición calculando la distancia euclidiana recorrida y se detiene al alcanzar el umbral configurado. Los resultados obtenidos fueron:

- Desplazamiento lineal de 1 metro: error promedio de 5-10 cm
- Giro de 90°: error promedio de 2-5°
- Trayectoria en cuadrado: cierre del bucle con desviación visible pero aceptable
- Trayectoria en S: ejecutada correctamente combinando velocidad lineal y angular simultáneas

Estos resultados confirman que el control en lazo cerrado con odometría ofrece una precisión considerablemente mayor que el control temporizado, siendo suficiente para las etapas de mapeo y navegación básica en interiores.

3.2. Integración y procesamiento del sensor Kinect

La integración del Kinect Xbox 360 como sensor sustituto de LiDAR resultó viable para los propósitos del proyecto. El nodo kinect_ros2 publicó correctamente los tópicos de imagen de profundidad y nube de puntos 3D a una frecuencia estable.

El nodo depthimage_to_laserscan con la configuración del archivo laserscan_config.yaml generó un LaserScan 2D en el tópico /scan a una frecuencia aproximada de 10 Hz, consistente con la frecuencia de captura configurada en el driver del Kinect. Sin embargo, el escaneo presentó lecturas erráticas conocidas como puntos fantasmas, originadas por reflexiones infrarrojas en superficies irregulares y patas de muebles.

La aplicación del nodo filtro_laserscan.py basado en el filtro de mediana de scipy.ndimage redujo considerablemente estas lecturas erráticas, publicando un tópico /scan_filtrado con mayor consistencia y estabilidad. La verificación visual en RViz2 confirmó que el patrón de escaneo filtrado representa correctamente el contorno del entorno a la altura del sensor, comportándose de manera equivalente a un sensor LiDAR 2D físico.

Se identificó que el uso de ros2 run directamente para lanzar el nodo del Kinect, en lugar del archivo pointcloud.launch.py incluido en el paquete, eliminó el problema de nodo duplicado que generaba conflictos de sincronización de timestamps en etapas posteriores.

3.3. Integración y procesamiento del sensor Kinect

Se realizaron múltiples sesiones de mapeo en dos entornos distintos para evaluar la calidad del mapa generado por slam_toolbox.

Mapa del laboratorio:

El laboratorio demostró ser un entorno desafiante para el sistema de

mapeo. La presencia de mesas, sillas y sus patas generó artefactos en abanico en el LaserScan, produciendo un mapa con zonas distorsionadas y paredes irregulares. Estos artefactos se originan porque el Kinect detecta múltiples objetos a diferentes distancias dentro del mismo ángulo de visión, lo que dificulta la correlación de escaneos del algoritmo SLAM. A pesar de múltiples intentos con diferentes estrategias de recorrido, la calidad del mapa en el laboratorio fue limitada.

Mapa del pasillo:

El pasillo contiguo al laboratorio produjo resultados significativamente mejores. Las paredes lisas y continuas del pasillo permitieron al LaserScan generar lecturas consistentes, facilitando la correlación de escaneos por parte de slam_toolbox. El mapa resultante mostró las paredes claramente definidas y el espacio libre correctamente delimitado, con pocos artefactos. Este resultado confirma que entornos con geometría simple son los más adecuados para sistemas de mapeo basados en cámara de profundidad.

Impacto del EKF:

La incorporación del filtro de Kalman extendido mediante el paquete robot_localization produjo una mejora notable en la consistencia del mapa. Sin EKF, el mapa tendía a presentar una ligera inclinación diagonal producto del drift acumulado de la odometría de ruedas. Con EKF, esta desviación se redujo considerablemente gracias a la corrección continúa aportada por los datos de la IMU, resultando en mapas con paredes más rectas y coherentes con la geometría real del entorno (ilustración 11 y 12).

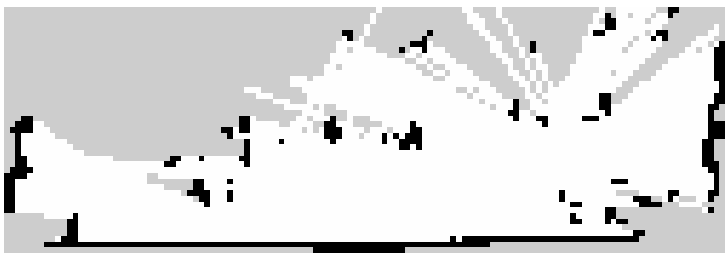


Ilustración 11. Mapa creado en laboratorio con EKF.

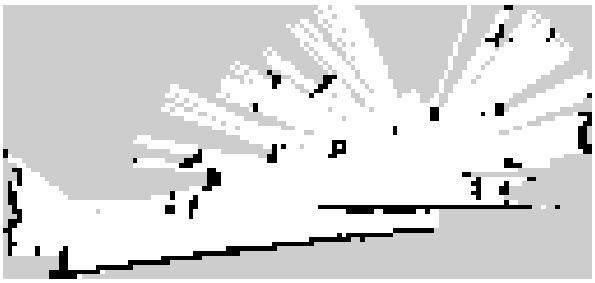


Ilustración 12. Mapa creado en laboratorio sin EKF.

3.3. Navegación autónoma

Las pruebas de navegación autónoma con Nav2 y AMCL demostraron el funcionamiento completo del sistema. Se realizaron las siguientes pruebas con resultados exitosos:

- Navegación a punto cercano (menos de 1 metro): completada correctamente.
- Navegación a punto lejano (más de 3 metros): completada correctamente.
- Navegación con giro de destino: el robot llegó al punto y se orientó en la dirección indicada.
- Navegación con obstáculo dinámico interpuesto: el costmap local detectó el obstáculo y Nav2 replanificó la ruta automáticamente.

Se identificaron los siguientes factores como críticos para el correcto funcionamiento del sistema de navegación:

- El uso de Cyclone DDS en lugar de FastDDS fue determinante para que AMCL procesara correctamente el LaserScan del Kinect, ya que FastDDS generaba problemas de sincronización de timestamps que impedían la localización.
- El valor de transform_tolerance: 0.5 tanto en AMCL como en el controller server fue necesario para tolerar la latencia introducida por el driver del Kinect.
- La transformada base_footprint → base_link fue indispensable para que el behavior server de Nav2 encontrara el frame base del robot.
- El relay de /cmd_vel a /commands/velocity fue necesario para la compatibilidad entre Nav2 y el driver del Kobuki.

IV.BIBLIOGRAFIA

- [1] Yujin Robot, "Kobuki Documentation," Read the Docs, 2020. [En línea]. Disponible en: https://iclebo-kobuki.readthedocs.io/en/latest/index.html.
- [2] Yujin Robot, "Kobuki User Guide," 2013. [En línea]. Disponible en: https://file.ncnynl.com/ros/Kobuki-UserGuide.pdf.
- [3] Open Robotics. (2026). Ubuntu (deb packages) — ROS 2 Documentation: Humble documentation. [En línea]. Disponible en: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html.
- [4] [Christopher J. Villa Alanis], "[SLAM_Documentation]", GitHub, 2026. [En línea]. Disponible en: https://github.com/rocketant124/SLAM_Documentacion.
- [5] Khoshelham, K.; Elberink, S. O. (2012), Accuracy and Resolution of Kinect Depth Data for Indoor Mapping Applications. *Sensors*, 12(2), 1437-1454. Disponible en: https://www.mdpi.com/1424-8220/12/2/1437.
- [6] F. Folch Company. (2023). Turtlebot 4 y ROS2: análisis de algoritmos SLAM y de navegación (Trabajo Fin de Máster, Universitat Politècnica de València). [En línea]. Disponible en: https://es.scribd.com/document/985428078/Turtlebot-4-y-ROS2-Analisis-de-Algoritmos-SLAM-y-De.
- [7] MathWorks. (2026). ekfSLAM - MATLAB Documentation. [En línea]. Disponible en: https://la.mathworks.com/help/nav/ref/ekfslam.html.
- [8] Open Navigation LLC. (2024). Nav2 - Your Autonomous Navigation Framework. [En línea]. Disponible en: https://nav2.org/
- [9] ROS Wiki. (2020). amcl - ROS Wiki. [En línea]. Disponible en: https://wiki.ros.org/amcl.

V. ANEXOS

Anexo A. Imagen ISO Ubuntu 22.04.5

El enlace oficial para la descarga de la imagen del sistema operativo es el siguiente:

https://releases.ubuntu.com/jammy/ubuntu-22.04.5-desktop-amd64.iso

Anexo B. Código completo “mover_metro.py”:

El código completo para la trayectoria de avanzar un metro en línea recta es el siguiente:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
import math

class MoverDistancia(Node):
    """
    Nodo de ROS 2 para desplazar el robot Kobuki una distancia recta fija
    utilizando retroalimentación de posición en lazo cerrado desde el tópico /odom.
    """
    def __init__(self):
        super().__init__('mover_distancia')

        # Publicador de comandos de velocidad a la base
        self.pub = self.create_publisher(Twist, '/commands/velocity', 10)

        # Suscriptor al flujo de odometría para el cálculo de distancia
        self.sub = self.create_subscription(Odometry, '/odom', self.odom_callback, 10)

        # Coordenadas origen
        self.x_inicio = None
```

```
        self.y_inicio = None

        # Meta de desplazamiento
        self.distancia_objetivo = 1.0  # metros
        self.get_logger().info(f'Avanzando {self.distancia_objetivo} metros...')

    def odom_callback(self, msg):
        """Calcula la distancia recorrida mediante la fórmula de distancia euclidiana 2D."""
        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y

        # Registra la posición inicial al recibir la primera lectura del sensor
        if self.x_inicio is None:
            self.x_inicio = x
            self.y_inicio = y

        # Distancia euclidiana recorrida respecto al punto de partida (m)
        distancia = math.sqrt((x - self.x_inicio)**2 + (y - self.y_inicio)**2)

        cmd = Twist()

        # Continua avanzando hasta que alcance el objetivo
        if distancia < self.distancia_objetivo:
            cmd.linear.x = 0.15
            self.get_logger().info(f'Distancia recorrida: {distancia:.3f}m')
        else:
            # El robot frena y se apaga el nodo
            cmd.linear.x = 0.0
            self.pub.publish(cmd)
            self.get_logger().info('¡Objetivo alcanzado!')
            rclpy.shutdown()
            return
```

```
        self.pub.publish(cmd)

def main():
    rclpy.init()
    node = MoverDistancia()
    rclpy.spin(node)

if __name__ == '__main__':
    main()
```

Anexo C. Demostraciones en video de trayectorias y movimientos básicos:

Link de YouTube: https://youtu.be/gGA1VmJdzmY.

Anexo D. Loop closure.

Al realizar un mapeo tele operado en pasillos largos (ida y vuelta), el mapa tiende a inclinarse o deformarse en el regreso, esto ocurre porque los pasillos son geométricamente homogéneos, lo que provoca que a medida que el robot avanza, el algoritmo de SLAM pierde la capacidad de corregir el error acumulado de las ruedas (la deriva). Al dar la vuelta y regresar, el mapa empieza a encimarse o torcerse porque el sistema cree que está en una posición ligeramente distinta a la real.

La solución: Al detectar un bucle (verificando mediante cámaras, LiDAR o sensores absolutos que la escena actual coincide con una pasada), el robot obtiene una medición de posición absoluta que corta el ciclo de errores acumulados (ilustración 13).

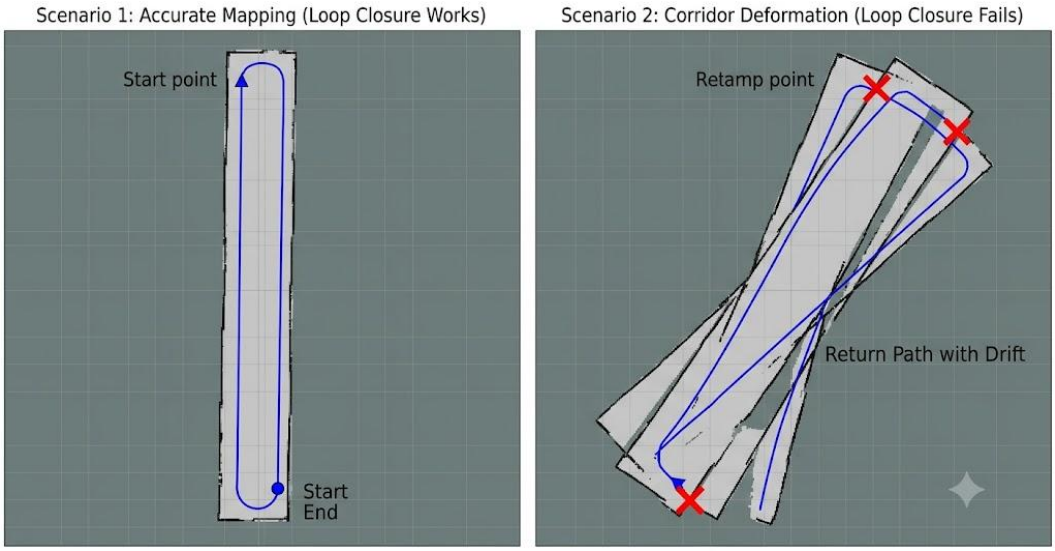


Ilustración 13. Imagen del funcionamiento de Loop Closure.

Aquí visualizamos el problema principal que estamos atacando. A la izquierda, un mapeo perfecto: el robot hace un bucle y regresa al mismo punto exacto. A la derecha, vemos el error de deriva (drift). Debido a que el pasillo es uniformemente largo y las ruedas resbalan ligeramente, el robot acumula error de rotación. Al regresar, el mapa se deforma y las paredes no se alinean, produciendo el error de deriva que el loop closure busca corregir.

Anexo E. Evidencias de mapeo.

Link YouTube: https://youtu.be/8vellvasD6s

Anexo F. Evidencias de navegación autónoma.

Link YouTube: https://youtu.be/YxljkDkqRZY